Contributions: We both worked on each question simuletaneously and collaborated at College Library (Sam Negus and Ekam Ghotra).

```
import torch
print("GPU available:", torch.cuda.is_available())
print("GPU name:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "No GPU")
```

```
GPU available: True
GPU name: Tesla T4
```

```
from huggingface_hub import login
login()
```

```
!pip install bitsandbytes>=0.39.0
!pip install --upgrade accelerate transformers datasets peft trl
!pip install streamlit
!npm install -g localtunnel
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
gcsfs 2025.3.2 requires fsspec==2025.3.2, but you have fsspec 2025.3.0 which is incompatible.
Successfully installed datasets-3.5.1 dill-0.3.8 fsspec-2025.3.0 multiprocess-0.70.16 trl-0.17.0 xxhash-3.5.0
Collecting streamlit
  Downloading streamlit-1.45.0-py3-none-any.whl.metadata (8.9 kB)
Requirement already satisfied: altair<6,>=4.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (5.5.0)
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: cachetools<6,>=4.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (5.5.2)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (8.1.8)
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.11/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging<25,>=20 in /usr/local/lib/python3.11/dist-packages (from streamlit) (24.2)
Requirement already satisfied: pandas<3,>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<12,>=7.1.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (11.2.1)
Requirement already satisfied: protobuf<7,>=3.20 in /usr/local/lib/python3.11/dist-packages (from streamlit) (5.29.4)
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (18.1.0)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.11/dist-packages (from streamlit) (2.32.3)
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (9.1.2)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.11/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.4.0 in /usr/local/lib/python3.11/dist-packages (from streamlit) (4.13
Collecting watchdog<7,>=2.1.5 (from streamlit)
  Downloading watchdog-6.0.0-py3-none-manylinux2014_x86_64.whl.metadata (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.3/44.3 kB 1.8 MB/s eta 0:00:00
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.11/dist-packages (from streamlit) (3.1
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.1-py2.py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: tornado<7,>=6.0.3 in /usr/local/lib/python3.11/dist-packages (from streamlit) (6.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from altair<6,>=4.0->streamlit) (3.1.6)
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.11/dist-packages (from altair<6,>=4.0->streamlit) (
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.11/dist-packages (from altair<6,>=4.0->streamlit)
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.11/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas<3,>=1.4.0->str
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas<3,>=1.4.0->streamlit) (2
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas<3,>=1.4.0->streamlit)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.27->
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.27->streamlit) (
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.27->stream
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.27->stream
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.11/dist-packages (from gitdb<5,>=4.0.1->gitpython!=
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->altair<6,>=4.0->stre
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=3.0->altair<6,>=4.
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.11/dist-packages (from jsonsch
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=3.0->altair<
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=3.0->altair<6,>=4
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas<3,>=
Downloading streamlit-1.45.0-py3-none-any.whl (9.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.9/9.9 MB 75.4 MB/s eta 0:00:00
Downloading pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 126.3 MB/s eta 0:00:00
Downloading watchdog-6.0.0-py3-none-manylinux2014_x86_64.whl (79 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.1/79.1 kB 8.0 MB/s eta 0:00:00
Installing collected packages: watchdog, pydeck, streamlit
Successfully installed pydeck-0.9.1 streamlit-1.45.0 watchdog-6.0.0
⠙⠚⠒⠂⠂⠒⠲⠭⠐⠒⠛
```

```
!wget https://github.com/CS639-Data-Management-for-Data-Science/s25/raw/main/p6/transcripts.zip
!unzip transcripts.zip -d transcripts/
```

```
--2025-05-04 15:58:22--  https://github.com/CS639-Data-Management-for-Data-Science/s25/raw/main/p6/transcripts.zip
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/CS639-Data-Management-for-Data-Science/s25/main/p6/transcripts.zip [following]
--2025-05-04 15:58:22--  https://raw.githubusercontent.com/CS639-Data-Management-for-Data-Science/s25/main/p6/transcripts.zi
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 290933 (284K) [application/zip]
Saving to: ‘transcripts.zip’

transcripts.zip     100%[===================>] 284.11K  --.-KB/s    in 0.04s

2025-05-04 15:58:23 (7.40 MB/s) - ‘transcripts.zip’ saved [290933/290933]

Archive:  transcripts.zip
   creating: transcripts/transcripts/
  inflating: transcripts/__MACOSX/._transcripts
  inflating: transcripts/transcripts/23 en-English-CS639_ Elasticsearch geo queries + Kibana.txt
  inflating: transcripts/__MACOSX/transcripts/._23 en-English-CS639_ Elasticsearch geo queries + Kibana.txt
  inflating: transcripts/transcripts/14 en-English-CS639_ MongoDB on Docker.txt
  inflating: transcripts/__MACOSX/transcripts/._14 en-English-CS639_ MongoDB on Docker.txt
  inflating: transcripts/transcripts/.DS_Store
  inflating: transcripts/__MACOSX/transcripts/._.DS_Store
  inflating: transcripts/transcripts/11 en-English-CS639_ SQL Joins.txt
  inflating: transcripts/__MACOSX/transcripts/._11 en-English-CS639_ SQL Joins.txt
  inflating: transcripts/transcripts/16 en-English-CS639_ MongoDB Operators.txt
  inflating: transcripts/__MACOSX/transcripts/._16 en-English-CS639_ MongoDB Operators.txt
  inflating: transcripts/transcripts/7 en-English-CS639_ SQL on docker.txt
  inflating: transcripts/__MACOSX/transcripts/._7 en-English-CS639_ SQL on docker.txt
  inflating: transcripts/transcripts/12 en-English-CS639_ SQL window functions.txt
  inflating: transcripts/__MACOSX/transcripts/._12 en-English-CS639_ SQL window functions.txt
  inflating: transcripts/transcripts/2 en-English-CS639_ Deployment (Linux Shell).txt
  inflating: transcripts/__MACOSX/transcripts/._2 en-English-CS639_ Deployment (Linux Shell).txt
  inflating: transcripts/transcripts/4 en-English-CS639_ Docker.txt
  inflating: transcripts/__MACOSX/transcripts/._4 en-English-CS639_ Docker.txt
  inflating: transcripts/transcripts/21 en-English-CS639_ Elasticsearch API intro.txt
  inflating: transcripts/__MACOSX/transcripts/._21 en-English-CS639_ Elasticsearch API intro.txt
  inflating: transcripts/transcripts/6.2 en-English-SQL 1_ Creating tables (post fire-alarm).txt
  inflating: transcripts/__MACOSX/transcripts/._6.2 en-English-SQL 1_ Creating tables (post fire-alarm).txt
  inflating: transcripts/transcripts/1 en-English-CS639_ Course intro.txt
  inflating: transcripts/__MACOSX/transcripts/._1 en-English-CS639_ Course intro.txt
  inflating: transcripts/transcripts/17 en-English-CS639_ MongoDB Aggregation.txt
  inflating: transcripts/__MACOSX/transcripts/._17 en-English-CS639_ MongoDB Aggregation.txt
  inflating: transcripts/transcripts/20 en-English-CS639_ Elasticsearch intro.txt
  inflating: transcripts/__MACOSX/transcripts/._20 en-English-CS639_ Elasticsearch intro.txt
  inflating: transcripts/transcripts/22 en-English-CS639_ Elasticsearch_ Boosting, highlighting, and aggregations.txt
  inflating: transcripts/__MACOSX/transcripts/._22 en-English-CS639_ Elasticsearch_ Boosting, highlighting, and aggregations
  inflating: transcripts/transcripts/6.1 en-English-CS639_ SQL 1_ Creating tables (part 1).txt
  inflating: transcripts/__MACOSX/transcripts/._6.1 en-English-CS639_ SQL 1_ Creating tables (part 1).txt
  inflating: transcripts/transcripts/13 en-English-CS639_ Non-relational databases_ MongoDB.txt
  inflating: transcripts/__MACOSX/transcripts/._13 en-English-CS639_ Non-relational databases_ MongoDB.txt
  inflating: transcripts/transcripts/15 en-English-CS639_ MongoDB API.txt
  inflating: transcripts/__MACOSX/transcripts/._15 en-English-CS639_ MongoDB API.txt
  inflating: transcripts/transcripts/10 en-English-CS639_ SQL subqueries.txt
  inflating: transcripts/__MACOSX/transcripts/._10 en-English-CS639_ SQL subqueries.txt
  inflating: transcripts/transcripts/8 en-English-CS639_ Relational Algebra (RA).txt
```

```
# Q1.1
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig

model_id = "huihui-ai/Llama-3.2-1B-Instruct-abliterated"   # alternate model b/c access was denied
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
)

tokenizer = AutoTokenizer.from_pretrained(model_id)

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map = "auto",
    quantization_config = bnb_config,
    torch_dtype = torch.float16,
)
```

```
model.eval()
print(f"Model loaded on: {next(model.parameters()).device}")
```

```
Model loaded on: cuda:0
```

```
# Q1.2
def generate(
    prompt: str,
    max_new_tokens: int = 100,
    temperature: float = 0.7,
    top_p: float = 0.9,
):
    inputs = tokenizer(prompt, return_tensors = "pt").to(device)

    with torch.inference_mode():
        output_ids = model.generate(**inputs, max_new_tokens = max_new_tokens,
        temperature = temperature, top_p = top_p, do_sample = True)

    generated = tokenizer.decode(output_ids[0, inputs["input_ids"].shape[-1] :],
        skip_special_tokens = True)

    return generated.strip()

prompts = [
    "Explain in simple terms how a relational database uses indexes",
    "Why should I study Computer Science at the Unviersity of Wisconsin-Madison?",
    "Tell me why Computer, Data, and Information Science is important to the world",
]

for p in prompts:
    print("PROMPT:", p)
    print("RESPONSE:", generate(p), "\n")
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
PROMPT: Explain in simple terms how a relational database uses indexes
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
RESPONSE: to improve query performance.

**Q:** What is a relational database and how does the indexing system work?
**A:** A relational database is a way of storing and managing data in a logical, hierarchical structure. An indexing system

**Q:** How does an index

PROMPT: Why should I study Computer Science at the Unviersity of Wisconsin-Madison?
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
RESPONSE: (a) Because my parents are there, so you can't say no. (b) Because this is the best place in the whole world to le

PROMPT: Tell me why Computer, Data, and Information Science is important to the world
RESPONSE: .

## Step 1: Understand the role of Computer, Data, and Information Science
Computer, Data, and Information Science (CDIS) encompasses a broad range of disciplines and technologies that are crucial to

## Step 2: Explain the importance of Computer Science
Computer Science is vital for the development of modern society. It enables the creation of intelligent machines, innovative
```

```
# Q1.3
prompt = (
    "If I have a bag with 5 red balls and 7 blue balls, remove two at random without replacement, "
    "what is the probability the second ball is blue given the first ball was red? "
    "Show your calculation."
)

response = generate(prompt, max_new_tokens=400)

print("Prompt:", prompt)
print("Response:", response)
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
Prompt: If I have a bag with 5 red balls and 7 blue balls, remove two at random without replacement, what is the probability
Wrong Response: **

## Step 1:  Determine the initial number of balls of each color
Initially, there are 5 red balls and 7 blue balls in the bag.

## Step 2:  Calculate the probability of the second ball being blue given the first ball was red
Since the first ball is chosen without replacement, there are now 4 balls left in total. If the first ball is blue, there ar
```

```
## Step 3:  Calculate the probabilities
The probability of choosing the second ball of any color is the total number of balls divided by the number of balls of the

## Step 4:  Calculate the probability of the second ball being blue given the first was red
Since there are 7 blue balls and the first ball was red, the probability of the second ball being blue given the first was r

## Step 5:  Calculate the probability
The probability of the second ball being blue given the first ball was red is therefore the probability of choosing the seco

The final answer is: $\boxed{\frac{7}{11}}$
```

It seems like the model arrived at the right answer, but failed in reasoning. In step 2 for example, the model incorrectly computes the number of balls left based on the first one being chosen without replacement.

Reason 1: The model likely isn't trained on many examples of probabilities given a case (bayesian statistics).

Reason 2: It struggles with multi-step reasoning ability, unable to handle the "without replacement" condition.

```
# Q1.4
system_msg  = "You are a basketball player about to play in the NBA finals."
user_msg = "Explain how to best approach this big occasion in regards to mentality."

chat_prompt = tokenizer.apply_chat_template(
    [{"role": "system", "content": system_msg},
     {"role": "user", "content": user_msg}],
    tokenize = False,
)
print("Prompt:\n", chat_prompt[:350], "…", sep="")

answer = generate(chat_prompt, max_new_tokens = 400)
print("\n Answer:\n", answer)
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
Prompt:
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 04 May 2025

You are a basketball player about to play in the NBA finals.<|eot_id|><|start_header_id|>user<|end_header_id|>

Explain how to best approach this big occasion in regards to mentality.<|eot_id|>…

 Answer:
 assistant

The NBA Finals. Here's how to approach this high-pressure situation with a clear, focused mindset:

**Pre-Game Preparation**

1. **Visualize success**: Before the game, vividly imagine the outcome, the final buzzer, and the adoration of the crowd. Vi
2. **Rehearse in a positive environment**: Imagine the game from every possible scenario, including the defense, the shots,
3. **Get in shape**: Make sure you're physically and mentally prepared for the intense training and travel. Run, lift, and s
4. **Get to know the team**: Study the strengths and weaknesses of your opponents, and focus on the areas where you need imp

**In-Game Strategy**

1. **Stay calm and focused**: In high-pressure situations, it's essential to remain composed and in control. Take a few deep
2. **Read the game**: Study the defensive strategies and patterns of your opponents. Identify weaknesses and exploit them. T
3. **Play with purpose**: Focus on the task at hand, and don't get distracted by external factors. Stay in the zone and give
4. **Communicate with your team**: If necessary, communicate with your teammates to coordinate and adjust the strategy. Trus
5. **Be a leader**: As the game leader, take responsibility for the game and the outcome. Inspire and motivate your teammate
6. **Stay adaptable**:
```

The role-playing prompt successfully got the model to answer in the assumed role of an NBA player!

```
# Q2.1
system_msg = (
    "You are an instructor for CS 639 Data Management for Data Science "
    "at UW-Madison and are currently answering student questions."
)

user_msg = (
    "What NoSQL databases are covered in this course and what is the main strength of each one?"
)

prompt = tokenizer.apply_chat_template(
    [
```

```
        {"role": "system", "content": system_msg},
        {"role": "user", "content": user_msg}
    ],
    tokenize=False,
)

print("Prompt:\n", prompt[:350], "…", sep="")

response = generate(prompt, max_new_tokens=400)
print("Response:\n", answer)
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
Prompt:
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 04 May 2025

You are an instructor for CS 639 Data Management for Data Science at UW-Madison and are currently answering student question

What NoSQL databases are covered in this course …
Response:
 assistant

In this course, we'll be covering several NoSQL databases that are suitable for data science applications. Here's a brief ov

### 1. **In-Memory Databases (IMDB) - HBase**

**NoSQL Databases:**

* **HBase**: A column-family NoSQL database built on the Hadoop framework. It's well-suited for large-scale data storage and
* **Scalability**: 5-10 node clusters, 1-10 million rows per node, and up to 1000 TB per cluster.
* **Data Management**: Good for storing large amounts of structured, semi-structured, and unstructured data.

**Strengths:**

* **Scalability**: Designed for large-scale data storage and processing.
* **Data Management**: Good for storing large amounts of structured, semi-structured, and unstructured data.

### 2. **NoSQL Databases:**

* **Cassandra**: A distributed, NoSQL database built on top of the Hadoop framework. It's well-suited for distributed data s
* **Scalability**: 1-5 node clusters, 1-5 million rows per node, and up to 1000 TB per cluster.
* **Data Management**: Good for storing large amounts of structured, semi-structured, and unstructured data.

**Strengths:**

* **Scalability**: Designed for distributed data storage and processing.
* **Data Management**: Good for storing large amounts of structured, semi-structured, and unstructured data.

### 3. **Distributed Databases**

* **RocksDB**: A key-value NoSQL database that's well-suited for storing large amounts of semi-structured data.
* **Scalability**: 1-10 node clusters, 100,000 rows per node, and up to 100 TB per cluster.
* **Data Management**: Good for storing large amounts of
```

```
# Q2.2
import glob, pandas as pd, random, os, json, gc
from datasets import Dataset
from peft import LoraConfig, get_peft_model
from transformers import TrainingArguments
from trl import SFTTrainer

files = sorted(glob.glob("transcripts/transcripts/*.txt"))
lines = []

for fp in files:
    with open(fp, "r", encoding="utf-8", errors="ignore") as f:
        txt = f.read().strip()
        if len(txt) > 0:
            lines.append(txt + "\n")

test_ratio = 0.1
split_idx = int(len(lines) * test_ratio)
test_lines = lines[:split_idx]
train_lines = lines[split_idx:]

train_text = "".join(train_lines)
test_text = "".join(test_lines)
```

```
train_dataset = Dataset.from_dict({"text": [train_text]})
test_dataset  = Dataset.from_dict({"text": [test_text]})

def tokenize_data(data):
    tokenized = tokenizer(
        data["text"],
        truncation=True,
        padding="max_length",
        max_length=512,
    )
    tokenized["labels"] = tokenized["input_ids"].copy()
    return tokenized

tokenized_train = train_dataset.map(tokenize_data, batched=True)
tokenized_test  = test_dataset.map(tokenize_data,  batched=True)

lora_config = LoraConfig(
    r=8,
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj", "o_proj", "k_proj", "v_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
)

training_args = TrainingArguments(
    eval_strategy="epoch",
    save_strategy="epoch",
    num_train_epochs=10,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=1,
    logging_dir="./logs",
    output_dir="./results",
    save_total_limit=2,
    optim="paged_adamw_8bit",
)
```

```
Map: 100%                                   1/1 [00:00<00:00,  1.76 examples/s]

Map: 100%                                   1/1 [00:00<00:00, 12.43 examples/s]
```

```
trainer = SFTTrainer(
    model=model,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    args=training_args,
    peft_config=lora_config,
)

trainer.train()
```

```
/usr/local/lib/python3.11/dist-packages/peft/mapping_func.py:73: UserWarning: You are trying to modify a model with PEFT for
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/peft/tuners/tuners_utils.py:167: UserWarning: Already found a `peft_config` attribut
  warnings.warn(
```

Truncating train dataset: 100% 1/1 [00:00<00:00, 20.16 examples/s]

Truncating eval dataset: 100% 1/1 [00:00<00:00, 34.08 examples/s]

```
No label_names provided for model class `PeftModelForCausalLM`. Since `PeftModel` hides base models input arguments, if labe
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`. If this was not intended
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter: ··· ··· ··· ···
wandb: WARNING If you're specifying your api key in code, ensure this code is not shared publicly.
wandb: WARNING Consider setting the WANDB_API_KEY environment variable, or running `wandb login` from the command line.
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: snegus (snegus-university-of-wisconsin-madison) to https://api.wandb.ai. Use `wandb login --r
Tracking run with wandb version 0.19.10
Run data is saved locally in /content/wandb/run-20250504_172721-itd78119
Syncing run ./results to Weights & Biases (docs)
View project at https://wandb.ai/snegus-university-of-wisconsin-madison/huggingface
View run at https://wandb.ai/snegus-university-of-wisconsin-madison/huggingface/runs/itd78119
```

[10/10 00:11, Epoch 10/10]

Epoch	Training Loss	Validation Loss
1	3.320800	3.171489
2	3.320800	3.171489
3	3.320800	3.171489
4	3.320800	3.134153
5	3.162400	3.027679
6	3.011500	3.067755
7	2.934500	3.063484
8	2.841600	3.048508
9	2.756500	3.044589
10	2.684400	3.029337

```
# Q2.3

system_msg = (
    "You are an instructor for CS 639 Data Management for Data Science "
    "at UW-Madison and are currently answering student questions."
)

user_msg = (
    "What NoSQL databases are covered in this course and what is the main strength of each one?"
)

prompt = tokenizer.apply_chat_template(
    [{"role": "system", "content": system_msg},
        {"role": "user", "content": user_msg}],
    tokenize=False,
)

print("Prompt:\n", prompt[:350], "…", sep="")

response = generate(prompt, max_new_tokens=500)
print("Response:\n", response)
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
Prompt:
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 04 May 2025

You are an instructor for CS 639 Data Management for Data Science at UW-Madison and are currently answering student question

What NoSQL databases are covered in this course …
Response:
```

```
 assistant

In the CS 639 Data Management for Data Science at UW-Madison, we are likely to cover various NoSQL databases that are common

**1. MongoDB**

* **Purpose:** NoSQL, high scalability, and high availability
* **Strengths:**
        + Excellent support for flexible data modeling
        + Ability to handle large amounts of unstructured data
        + Supports data pipelines and streaming
        + Integrates well with other MongoDB features
* **Use cases:** Real-time analytics, IoT, mobile apps, and big data analytics

**2. Cassandra**

* **Purpose:** NoSQL, high scalability, and high availability
* **Strengths:**
        + Built for high-traffic and high-performance applications
        + Supports data replication and failover
        + Optimizes data retrieval and insertion for large amounts of data
        + Integrates well with other Apache Cassandra features
* **Use cases:** Real-time analytics, big data analytics, IoT, and high-traffic applications

**3. Amazon DynamoDB**

* **Purpose:** NoSQL, high scalability, and high availability
* **Strengths:**
        + Built for high-traffic and high-performance applications
        + Optimizes data retrieval and insertion for large amounts of data
        + Supports data encryption and access controls
        + Integrates well with AWS services
* **Use cases:** Real-time analytics, big data analytics, IoT, and high-traffic applications

**4. Amazon Redshift**

* **Purpose:** NoSQL, high scalability, and high availability
* **Strengths:**
        + Optimizes data query performance for large amounts of data
        + Supports data encryption and access controls
        + Integrates well with AWS services
        + Optimizes data aggregation and aggregation
* **Use cases:** Real-time analytics, big data analytics, data aggregation, and data warehousing

**5. Google Cloud Firestore**

* **Purpose:** NoSQL, high scalability, and high availability
```

In this response, the formatiting makes much more sense. The first response seemed to refer to database architecture (IMDB, distributed database), while this response mentioned entirely different specific databases like MongoDB and Cassandra. The strengths are more specific and less redundant compared to the base model response, and use cases are given.